
Parallel and Distributed Computing

Maurice Oliver Y. Dela Cruz

Chapter 5: Shared-Memory Programming with OpenMP



Chapter 3: Shared-Memory Programming with OpenMP

Objective:

Explore OpenMP as a high-level API for shared-memory parallel programming.

Key Concepts

OpenMP constructs:
`#pragma omp parallel`, `#pragma omp for`.

Variable scoping:
private, shared,
and reduction
clauses.

Loop scheduling
and handling
data
dependencies.

— What is OpenMP?

OpenMP is an API for shared-memory parallel programming. The “MP” in OpenMP stands for “*multiprocessing*,” a term that is synonymous with shared-memory parallel computing. Thus,

— What is OpenMP?

OpenMP is designed for systems in which each thread or process can potentially have access to all available memory, and, when we're programming with OpenMP, we view our system as a collection of cores or CPUs, all of which have access to main memory.

What is OpenMP?

OpenMP provides what's known as a “directives-based” shared-memory API. In C and C++, this means that there are special preprocessor instructions known as *pragmas*. Pragmas are typically added to a system to allow behaviors that aren't part of the basic C specification.

What is OpenMP?

Pragmas in C and C++ start with

```
#pragma
```

We put the pound sign, #, and like other preprocessor directives, we shift the remainder of the directive so that it is aligned with the rest of the code.

— What is OpenMP?

Pragmas, by default, one line in length, so if a pragma won't fit on a single line, the newline needs to be “escaped”—that is, preceded by a backslash n. The details of what follows the #pragma depend entirely on which extensions are being used.

Example: OpenMP “Hello World” program

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Hello(void); /* Thread function */

int main(int argc, char* argv[]) {
    /* Get number of threads from command line */
    int thread_count = strtol(argv[1], NULL, 10);

    # pragma omp parallel num threads(thread_count)
    Hello();

    return 0;
} /* main */
```

Example: OpenMP “Hello World” program

```
void Hello(void) {  
    int my_rank = omp_get_thread_num();  
    int thread_count = omp_get_num_threads();  
  
    printf("Hello from thread %d of %d\n", my_rank, thread_count);  
  
} /* Hello */
```

– **Compiling and Execution**

To compile this with gcc we need to include the -fopenmp option:

```
$ gcc -g -Wall -fopenmp -o omp_hello omp_hello.c
```

– **Compiling and Execution**

To run the program, we specify the number of threads on the command line. For example, we might run the program with four threads and type

```
$ ./omp hello 4
```

– **Compiling and Execution**

The output might be

```
Hello from thread 0 of 4
```

```
Hello from thread 1 of 4
```

```
Hello from thread 2 of 4
```

```
Hello from thread 3 of 4
```

– **Compiling and Execution**

However, it should be noted that the threads are competing for access to stdout, so there's no guarantee that the output will appear in thread-rank order.

– **Compiling and Execution**

For example, the output might also be:

```
Hello from thread 1 of 4
```

```
Hello from thread 2 of 4
```

```
Hello from thread 0 of 4
```

```
Hello from thread 3 of 4
```


– **Compiling and Execution**

Or might also be:

Hello from thread 3 of 4

Hello from thread 1 of 4

Hello from thread 2 of 4

Hello from thread 0 of 4

– **Compiling and Execution**

If we want to run the program with just one thread, we can type:

```
$ ./omp hello 1
```

and we would get the output

```
Hello from thread 0 of 1
```

#pragma

The *#pragma omp parallel* directive initiates parallel execution of a structured block.

The master thread executes the code before the parallel directive.

#pragma

Each thread has a unique rank or ID, retrievable using *omp_get_thread_num()*.

The total number of threads in a team can be obtained using *omp_get_num_threads()*.

#pragma

The *#pragma omp for* directive divides loop iterations among threads in an existing team. The run-time system must determine the total number of iterations before execution. Loop-carried dependencies must be identified and eliminated by the programmer.

Parallelizing Loops with OpenMP

OpenMP allows for the parallelization of loops using the `for` directive, which partitions iterations among threads. Proper loop structure and absence of loop-carried dependencies are essential for successful parallelization.

– Variable Scope and Reduction in OpenMP

Variable scope in OpenMP determines accessibility among threads, with shared and private scopes being common. Reduction variables are special cases that require careful handling to combine results from multiple threads.

– Variable Scope and Reduction in OpenMP

Variables defined before an OpenMP directive typically have shared scope. Loop variables in for constructs are private to each thread.

The reduction clause is used to specify reduction variables and operators.

– Scheduling and Performance in OpenMP

OpenMP provides various scheduling options for loop iterations, affecting performance based on how iterations are assigned to threads. The schedule clause allows for different strategies to optimize execution.

– Scheduling and Performance in OpenMP

Scheduling types include static, dynamic, guided, auto, and runtime.

Static scheduling assigns iterations before execution, while dynamic and guided assign them on-the-fly.

Example:

In our example, we already know how to obtain the default schedule: we just add a *parallel for* directive with a *reduction* clause:

```
sum = 0.0;
# pragma omp parallel for num threads(thread count) n
    reduction(+:sum)
for (i = 0; i <= n; i++)
    sum += f(i);
```

Example:

To get a cyclic schedule, we can add a schedule clause to the parallel for directive:

```
sum = 0.0;
# pragma omp parallel for num threads(thread count) n
    reduction(+:sum) schedule(static,1)
for (i = 0; i <= n; i++)
    sum += f(i);
```

Example:

In general, the schedule clause has the form:

```
schedule(<type> [, <chunksize>])
```

The chunksize is a positive integer. In OpenMP parlance, a chunk of iterations is a block of iterations that would be executed consecutively in the serial loop.

– Cache Coherence and False Sharing

Cache coherence issues can arise in multi-threaded programs, particularly with false sharing, where threads access different variables that reside in the same cache line. This can degrade performance significantly.

– Cache Coherence and False Sharing

False sharing occurs when threads modify different variables in the same cache line, causing unnecessary cache coherence traffic.

Static variables in functions can lead to thread safety issues if not managed properly.

Identifying and Avoiding Race Conditions

Race conditions can lead to incorrect program behavior, even if the output appears correct during testing.

- Developers must identify potential race conditions in their code.

Identifying and Avoiding Race Conditions

- Correct output during testing does not guarantee the absence of race conditions.
- Developers must be vigilant in identifying shared resource access patterns.

—

Thank you for listening.

